

INTERNSHIP PROJECT REPORT

Honeypot Login Page

A controlled demonstration of login-attempt monitoring using a decoy web interface

Submitted by	Pothula Yashwanth
Email	pothulayashwanth55@gmail.com
College	Presidency University
Semester	5th
Internship ID	HH8-9014638
Submission Date	26 July 2026

Prepared for internship evaluation

This report documents a local, educational honeypot demonstration. It is not connected to a real authentication system.

1. Introduction

A honeypot is a deliberately controlled system designed to observe suspicious or unauthorized activity. This project presents a small web-based decoy called an Employee Portal. It resembles a login form, but it does not grant access to any real service. Instead, each submitted attempt is recorded locally for educational security analysis.

2. Objective

The objective is to demonstrate how a controlled decoy login page can collect useful evidence of login attempts, including the submitted username, source IP address, browser information, and time of submission. The project also demonstrates responsible use: it is intended only for an isolated local test environment and must never be connected to genuine user accounts or production services.

3. Technologies Used

- **Python:** Runs the application logic and coordinates the web routes and logging process.
- **Flask:** Provides the lightweight web server and routes for the login page and the log-viewing page.
- **SQLite:** Stores captured test attempts locally in a compact database file.
- **HTML and CSS:** Present the Employee Portal form and the log table in a browser.
- **Web browser:** Used to access the local URL at 127.0.0.1:5000.

4. Working Principle

The application starts a Flask development server on the local computer. A visitor opens the Employee Portal form in a browser and submits a username and password. The system intentionally rejects the sign-in request, while recording selected request details in the local SQLite log. The administrator can then view the stored entries through the local `/admin/logs` route.

Execution Flow

- Start the Python application and open the local server address.
- Display the controlled Employee Portal login form.
- Receive a test sign-in attempt and collect request metadata.
- Store the observation in SQLite and show an invalid-login response.
- Review the recorded events on the local log page.

5. Detailed Explanation

Application startup: Running `honeypot_login_page.py` starts the Flask development server and displays the local URLs for the login form and administrator log view.

Login route: The form accepts submitted values solely as test data. It does not authenticate the visitor or grant access to any protected resource.

Data collection: For each attempt, the application records the timestamp, IP address, username, submitted password, and user-agent string. The user-agent helps identify the browser and operating-system context.

Storage and review: SQLite keeps the entries on the local system. The log page displays them in a table for analysis. Password values in the report screenshot have been masked for privacy.

Safety boundary: This demonstration is a decoy for local learning. It should remain isolated and must not collect real credentials or be used as a production authentication service.

6. Features

- Simple browser-based Employee Portal login interface.
- Local Flask server with separate login and log-viewing URLs.
- Local recording of login-attempt metadata in SQLite.
- Timestamped log table for reviewing captured test events.
- Consistent invalid-login response that preserves the decoy behaviour.

7. Advantages

- Provides a practical introduction to web security monitoring and request logging.
- Uses lightweight, widely available Python tools that are easy to demonstrate.
- Keeps all activity local, making the exercise suitable for a controlled academic setting.
- Shows how browser metadata and timing can support investigation of test events.

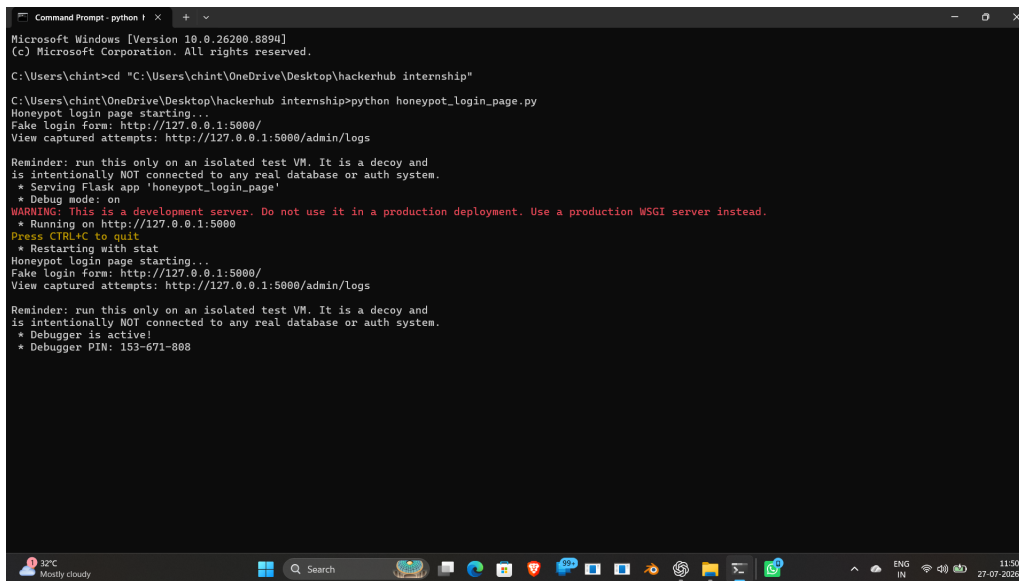
8. Applications

- Cybersecurity training labs and classroom demonstrations.
- Controlled research on web login behaviour and logging workflows.
- Practice environments for learning Flask, SQLite, HTTP requests, and audit records.

9. Implementation Evidence

The following screenshots show the application being started, the login interface, the intentional rejection message, and the recorded local events.

9. Implementation Evidence



```
Microsoft Windows [Version 10.0.26200.8894]
(c) Microsoft Corporation. All rights reserved.

C:\Users\chint>cd "C:\Users\chint\OneDrive\Desktop\hackerhub internship"

C:\Users\chint\OneDrive\Desktop\hackerhub internship>python honeypot_login_page.py
Honeypot login page starting...
Fake login form: http://127.0.0.1:5000/
View captured attempts: http://127.0.0.1:5000/admin/logs

Reminder: run this only on an isolated test VM. It is a decoy and
is intentionally NOT connected to any real database or auth system.
* Serving Flask app 'honeypot_login_page'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
Honeypot login page starting...
Fake login form: http://127.0.0.1:5000/
View captured attempts: http://127.0.0.1:5000/admin/logs

Reminder: run this only on an isolated test VM. It is a decoy and
is intentionally NOT connected to any real database or auth system.
* Debugger is active!
* Debugger PIN: 153-671-808
```

Figure 1. The honeypot application starts successfully and displays the local login and log-view URLs.

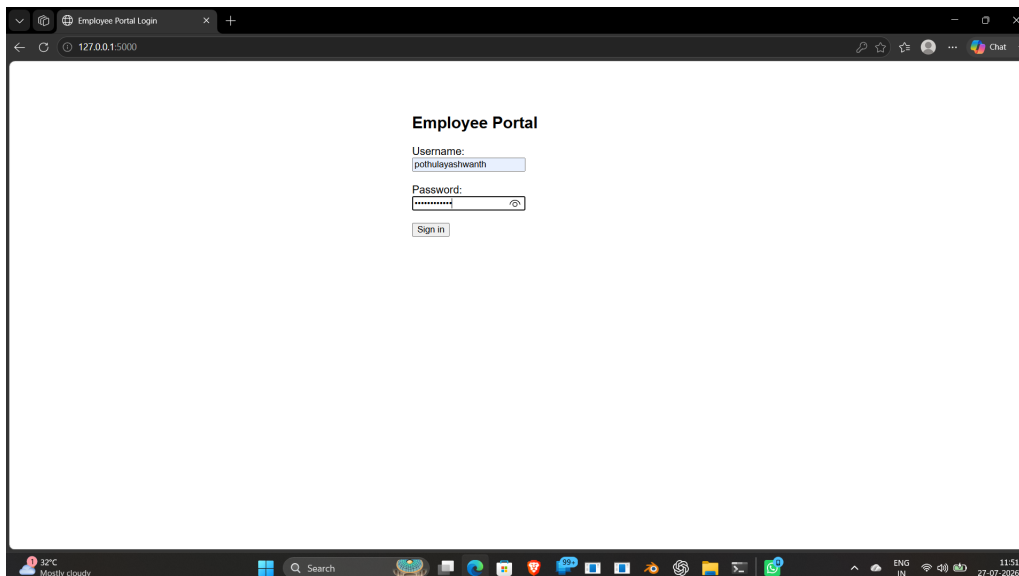


Figure 2. The Employee Portal decoy login interface is displayed in the browser.

9. Implementation Evidence (continued)

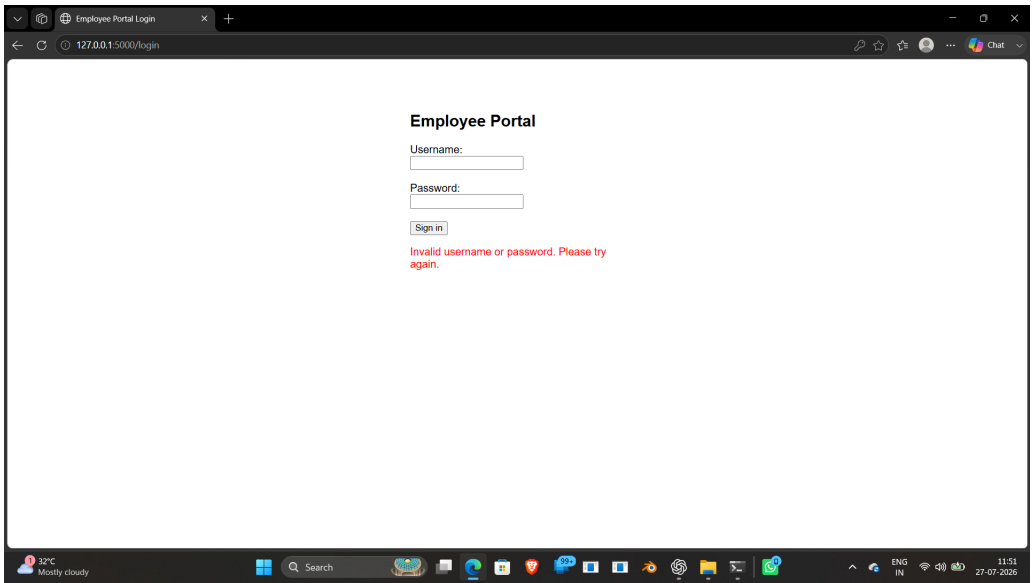


Figure 3. The portal returns an invalid username or password message after a test sign-in attempt.

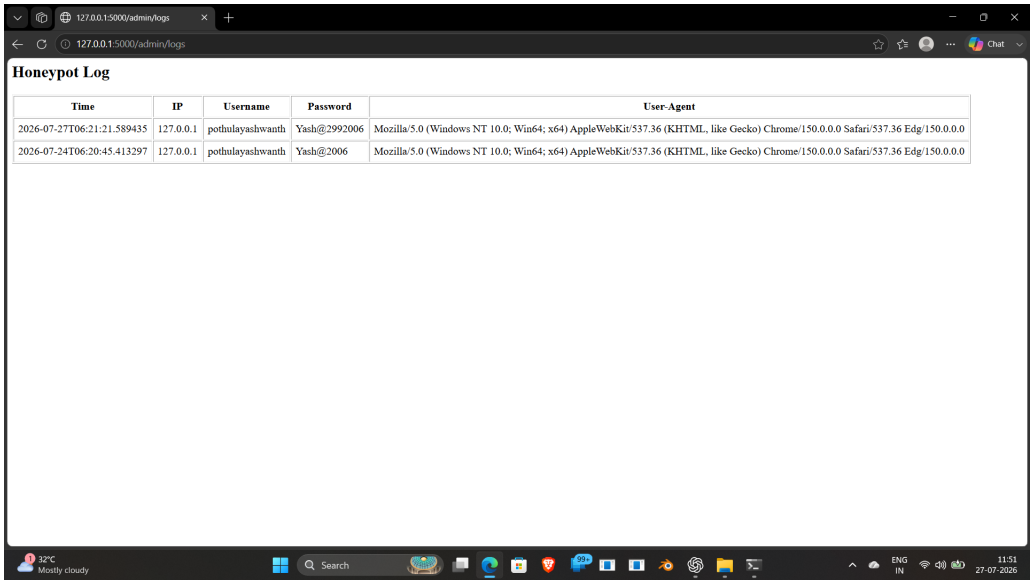


Figure 4. The local Honeypot Log lists the captured test entries. Password values are masked in this report for privacy.

10. Conclusion

The Honeypot Login Page was successfully implemented and demonstrated in a local environment. It shows the complete journey from application startup to decoy login submission, intentional rejection, and review of timestamped logs. The project strengthens understanding of Flask web applications, SQLite-based logging, and the security value of controlled deception systems. The implementation should remain a local educational demonstration and must not be connected to any real user accounts or production environment.